

INTERNSHIP PROJECT REPORT

AI-POWERED RESUME PARSER AND ATS ANALYZER

TASK 2 – RESUME PARSER

TABLE OF CONTENTS

1. Introduction
2. Internship Objectives
3. Problem Definition
4. Requirement Analysis
5. Existing Approach
6. Proposed Solution
7. System Architecture
8. Technology Stack
9. Project Modules
10. Implementation Details
11. ATS Scoring and Job Matching
12. Data Handling
13. Testing and Validation
14. Results
15. Challenges and Solutions
16. Security and Reliability Considerations
17. Limitations
18. Future Scope
19. Conclusion
20. References
21. Appendix A – Project Structure
22. Appendix B – Basic Execution

1. INTRODUCTION

Recruitment processes often involve reviewing a large number of resumes that differ in structure, formatting and terminology. Manual screening can require significant time and may lead to inconsistent evaluation. Applicant Tracking Systems (ATS) are commonly used to organize applications and identify resumes that are relevant to a particular job.

This project implements an internship-level resume processing system that accepts resume documents, extracts their textual content and organizes important candidate information into structured categories. The extracted information can then be used for resume scoring, ATS-oriented analysis, job matching and interview preparation.

The project demonstrates the practical use of Python, Flask, document processing, modular programming, HTML templates, JSON data handling and testing.

2. INTERNSHIP OBJECTIVES

- Understand a practical software development workflow through an AI-oriented project.
- Develop a modular Python application for resume/document processing.
- Extract text from PDF resumes.
- Identify contact details, education, experience, skills, projects and certifications.
- Develop resume analysis and resume scoring components.
- Implement ATS-oriented scoring and job-description matching.
- Provide a browser-based interface using Flask and HTML templates.
- Use a Python virtual environment and requirements.txt for dependency management.
- Create test modules for important components.
- Maintain a clean and extensible project structure.

3. PROBLEM DEFINITION

The main problem addressed by the project is the conversion of unstructured resume content into structured information that can be reused for automated analysis. Resumes may contain different section names, layouts and writing styles, making direct processing difficult.

The system therefore aims to:

- Accept resume PDF files.
- Extract readable text from the document.
- Identify important resume sections and candidate attributes.
- Store the extracted information in a structured representation.
- Analyze the resume against defined criteria.
- Calculate an ATS-oriented score.
- Compare resume information with a job description.
- Display useful results through a web interface.

4. REQUIREMENT ANALYSIS

4.1 Functional Requirements

- Resume upload and processing.
- PDF/text extraction.
- Contact information extraction.
- Education extraction.
- Experience extraction.
- Skills extraction.
- Project extraction.
- Certification extraction.
- Resume analysis and scoring.
- ATS scoring.
- Job-description matching.
- Interview-question generation support.
- Structured JSON output.
- HTML-based result presentation.

4.2 Non-Functional Requirements

- Maintainable modular architecture.
- Simple and understandable user interface.
- Reasonable processing time for normal-sized resumes.
- Reliable handling of expected input formats.
- Reusable and testable components.
- Secure handling of uploaded documents.
- Easy dependency installation and environment reproduction.

5. EXISTING APPROACH

A traditional recruitment workflow may depend on manual resume review or basic keyword filtering. Manual review can provide human context but becomes difficult when the number of applications is large. Simple keyword filtering is faster but may not provide structured information or meaningful job-specific analysis.

Existing Method	Limitation
Manual resume screening	Time-consuming and difficult to scale.
Basic keyword filtering	May miss related terms and context.
Unstructured document review	Information is difficult to reuse automatically.
Manual job comparison	Requires repeated effort for every job description.

6. PROPOSED SOLUTION

The proposed solution uses a modular pipeline in which a resume is uploaded through a Flask web interface, converted into text and passed through dedicated extraction components. The extracted profile is then used by analysis, scoring and job-matching modules.

Resume PDF → Text Extraction → Section Extraction → Structured Resume → Analysis → ATS Score / Job Match → Results

This design separates document processing from analysis and presentation, making the application easier to maintain and extend.

7. SYSTEM ARCHITECTURE

Based on the implemented project structure, the system can be viewed as the following layers:

Layer	Components	Purpose
Presentation Layer	templates/index.html, templates/result.html	User interaction, upload form and result display.
Application Layer	app.py	Flask routes and request/response handling.
Parsing Layer	parser.py, extract_pdf_text.py, text_extractor.py	Resume text extraction and parsing coordination.
Extraction Layer	extractors/*.py	Extract contact, education, experience, skills, projects and certifications.
AI/Analysis Layer	ai/resume_analyzer.py, ai/resume_score.py, ai/interview_generator.py	Resume analysis, scoring and interview support.
ATS Layer	ats/ats_score.py, ats/job_matcher.py	ATS-oriented scoring and job matching.
Data/Output Layer	data/skills.csv, output/parsed_resume.json, resumes/	Reference data, structured output and sample documents.
Testing Layer	test *.py	Component-level validation.

[Insert a simple system architecture diagram here]

8. TECHNOLOGY STACK

Technology / Tool	Purpose	Use in Project
Python 3.14.6	Programming language	Backend logic, parsing and analysis.
Flask	Web framework	Local web application and routing.
HTML	Frontend technology	Upload and result pages.
PDF processing library	Document processing	Extracting text from resume PDFs.
JSON	Data format	Storing structured parsed resume information.
CSV	Reference data	Skills reference list.
pytest / Python test modules	Testing	Validation of project components.
Python virtual environment	Dependency isolation	Project-specific environment.
Visual Studio Code	Development environment	Project development and testing.

9. PROJECT MODULES

File / Folder	Description
app.py	Main Flask application entry point and web request handling.
parser.py	Coordinates resume parsing and structured output generation.
extract_pdf_text.py	Handles PDF text extraction.
extractors/text_extractor.py	Supports text extraction/processing.
extractors/contact_extractor.py	Extracts contact information.
extractors/education_extractor.py	Extracts education details.
extractors/experience_extractor.py	Extracts work experience.
extractors/skills_extractor.py	Identifies relevant skills.
extractors/projects_extractor.py	Extracts project information.
extractors/certifications_extractor.py	Extracts certifications.
ai/resume_analyzer.py	Performs resume-oriented analysis.
ai/resume_score.py	Calculates resume score based on implemented criteria.
ai/interview_generator.py	Supports interview-question generation.
ats/ats_score.py	Calculates ATS-oriented score.
ats/job_matcher.py	Compares resume content with a job description.
data/skills.csv	Reference data for skills.
output/parsed_resume.json	Structured parsed resume output.
templates/index.html	Resume upload interface.
templates/result.html	Processed result display.
tests/*.py	Component-level tests.

10. IMPLEMENTATION DETAILS

10.1 Resume Upload and PDF Processing

The Flask application provides a browser-based workflow for submitting a resume. The uploaded PDF is processed to obtain machine-readable text. This text becomes the input for the parsing and extraction components.

10.2 Information Extraction

The project separates extraction into multiple files. This allows each information category to be processed independently and makes it easier to improve one extractor without changing the entire application.

10.3 Structured Resume Output

The parsed information is represented in structured JSON. A simplified representation is:

```
{
  "contact": {},
  "education": [],
  "experience": [],
  "skills": [],
  "projects": [],
```

```
"certifications": []  
}
```

10.4 Flask Web Interface

The Flask application runs locally on the development server and provides the web interface through HTML templates. The project screenshot confirms the application is running at <http://127.0.0.1:5000> during local development.

10.5 Project Environment

The project uses a dedicated virtual environment. The environment information visible in the development setup indicates Python 3.14.6. Dependencies are maintained separately so that the project can be reproduced without relying on the global Python installation.

11. ATS SCORING AND JOB MATCHING

The ATS component evaluates the resume according to criteria implemented in the project. These criteria may include relevant skills, keyword coverage, resume-section completeness and alignment with a target role.

The job matcher compares extracted resume information with the job description used by the application. The resulting score or match percentage is an application-specific indicator and should not be interpreted as an official score used by a real recruitment platform.

A conceptual scoring model is:

ATS Score = weighted combination of skill relevance + keyword coverage + section completeness + job alignment

12. DATA HANDLING

The project uses lightweight files for development and demonstration. The `skills.csv` file provides reference data for skill processing. The `resumes` folder contains sample PDF documents. The output folder contains `parsed_resume.json`, which represents structured processing results.

Data Item	Purpose
<code>data/skills.csv</code>	Reference list for skills.
<code>resumes/output.pdf</code>	Sample/output PDF document.
<code>resumes/resume.pdf</code>	Resume input sample.
<code>resumes/sample_resume.pdf</code>	Sample resume for testing.
<code>output/parsed_resume.json</code>	Structured parsed resume output.

For a production system, these files could be replaced or supplemented with a secure database and protected document storage.

13. TESTING AND VALIDATION

The project contains multiple test files covering important extraction and analysis components. The visible project structure includes tests for AI, ATS, certifications, contact, education, experience, projects, skills and text processing.

Test Area	Validation Goal	Expected Result
Text extraction	Check whether text is extracted from a resume.	Readable text is returned.
Contact extraction	Check contact patterns.	Contact fields are identified.
Education extraction	Check education processing.	Education details are structured.
Experience extraction	Check experience processing.	Experience details are structured.
Skills extraction	Check skill identification.	Relevant skills are detected.
Project extraction	Check project section processing.	Projects are extracted when present.
Certification extraction	Check certification processing.	Certifications are extracted when present.
ATS module	Check ATS scoring logic.	A consistent score is produced.
AI modules	Check analysis/score/interview functionality.	Expected analysis is returned.
End-to-end workflow	Check Flask application flow.	Resume can be submitted and results displayed.

14. RESULTS

The implemented project demonstrates a complete internship-level resume-processing workflow. The project structure includes the Flask application, parsing utilities, separate extraction modules, AI analysis modules, ATS modules, reference data, HTML templates, sample resumes, structured JSON output and test files.

The application was successfully launched locally using Flask. The development terminal shown in the project setup reports that the Flask application is serving requests at `http://127.0.0.1:5000` with debug mode disabled.

The main technical result is the transformation of an unstructured resume document into structured candidate information that can be used for resume analysis, scoring, ATS evaluation, job matching and interview preparation.

[Insert your actual application output screenshots here]

15. CHALLENGES AND SOLUTIONS

Challenge	Solution / Approach
Different resume layouts	Used separate extraction modules so section-specific logic can be maintained independently.
Unstructured resume text	Converted extracted content into structured categories.
Large number of responsibilities	Separated Flask, parser, extractors, AI and ATS components.
Dependency management	Used a project-specific virtual environment and <code>requirements.txt</code> .
Validation of components	Maintained individual test files for important modules.
Future scalability	Kept data, processing and presentation responsibilities separated.

16. SECURITY AND RELIABILITY CONSIDERATIONS

- Validate uploaded file extensions and file sizes.
- Use secure filenames when storing uploaded documents.
- Avoid executing any uploaded file content.
- Keep sensitive resume information out of application logs.
- Protect API keys and secrets through environment variables.
- Use authentication and authorization for recruiter-facing deployments.
- Use a production WSGI server instead of Flask's development server for deployment.
- Delete or securely manage uploaded resumes when they are no longer required.

17. LIMITATIONS

- PDF extraction accuracy depends on the document's quality and layout.
- Rule-based or keyword-based extraction may not understand every semantic variation.
- ATS scoring depends on the criteria implemented by the application.
- Job matching may be limited when synonyms or semantic relationships are not explicitly handled.
- The current project is primarily a local prototype and does not include a production database.
- Scanned/image-based resumes may require OCR for reliable extraction.

18. FUTURE SCOPE

- Use transformer embeddings for semantic job matching.
- Add LLM-assisted resume improvement recommendations.
- Support DOCX and image-based resumes using OCR.
- Create a recruiter dashboard for candidate ranking.
- Integrate PostgreSQL or MySQL for persistent data management.
- Add authentication and role-based access control.
- Support resume versioning and job-specific optimization.
- Add skill-gap analytics and hiring insights.
- Deploy the application to a cloud platform using a production WSGI server.
- Introduce CI/CD, automated quality checks and monitoring.

19. CONCLUSION

The AI-Powered Resume Parser and ATS Analyzer demonstrates how a practical recruitment-oriented application can be developed using Python and Flask. The project combines PDF processing, modular information extraction, structured JSON output, AI-oriented analysis, ATS scoring, job matching and a browser-based interface.

The modular design improves maintainability and provides a foundation for future AI capabilities. The project also provided practical experience in Python development, Flask web applications,

document processing, virtual environments, testing, structured data handling and technical documentation.

Overall, the project successfully demonstrates an internship-level implementation of an intelligent resume analysis workflow and provides a strong base for further development into a production-ready recruitment assistance platform.

20. REFERENCES

- Python Documentation — <https://docs.python.org/>
- Flask Documentation — <https://flask.palletsprojects.com/>
- Python Package Index (PyPI) — <https://pypi.org/>
- General technical references related to Natural Language Processing, PDF processing and Applicant Tracking Systems.
- Project source code, test files and supporting data developed during the internship.

APPENDIX A – PROJECT STRUCTURE

```
RESUME_PARSER/  
├── ai/  
│   ├── __init__.py  
│   ├── interview_generator.py  
│   ├── resume_analyzer.py  
│   └── resume_score.py  
├── ats/  
│   ├── ats_score.py  
│   └── job_matcher.py  
├── data/  
│   └── skills.csv  
├── extractors/  
│   ├── certifications_extractor.py  
│   ├── contact_extractor.py  
│   ├── education_extractor.py  
│   ├── experience_extractor.py  
│   ├── projects_extractor.py  
│   ├── skills_extractor.py  
│   └── text_extractor.py  
├── models/  
├── output/  
│   └── parsed_resume.json  
├── resumes/  
│   ├── output.pdf  
│   ├── resume.pdf  
│   └── sample_resume.pdf  
├── templates/  
│   ├── index.html  
│   └── result.html  
├── app.py  
└── extract_pdf_text.py
```

- |— parser.py
- |— README.md
- |— requirements.txt
- |— output1.png
- |— output2.png
- |— test_*.py

APPENDIX B – BASIC EXECUTION

The project can be executed in a Python virtual environment using the following commands:

```
python -m venv venv
venv\Scripts\activate
pip install -r requirements.txt
python app.py
```

The local Flask application is expected to be available at:

<http://127.0.0.1:5000>

Tests can be executed using the project's test setup. If pytest is configured, use:

```
python -m pytest
```